Performance Engineering of a Spark Pipeline for NYC Yellow Taxi Analytics

ComPar 2026/27 – Project 1 – Group [TODO: number]

Diogo Oliveira
André Nunes
77474
77077

ABSTRACT

We profiled and optimised a Kotlin Spark pipeline over the 146 M trips of the 2015 NYC Yellow Taxi data on a small Spark standalone cluster. The baseline lost its time in repeated scans, shuffles of whole trip rows and many small stages. Removing them one version at a time brought the five operations from 307 to 112 s (2.7×), and the operations that shuffled whole rows stopped spilling and need a fraction of the memory.

KEYWORDS

Apache Spark, query optimisation, profiling, shuffle, scalability

1 INTRODUCTION

We use the 2015 NYC TLC Yellow Taxi trip records [2]: 12 monthly Parquet files (2.0 GB) with 146,039,231 trips and 19 columns, plus the zone lookup table (265 rows). We started on 2024 and moved to 2015 because it has 3.5 times more trips, enough for shuffles and spills to show up on our small cluster. We implemented the mandatory **M1** (ingestion and cleaning), **M2** (temporal demand per borough, day and hour) and **M3** (monthly zone statistics with windows), and the optional **O1** (fare anomalies with $\mu + 3\sigma$ per zone and hour) and **O4** (trip-type classification).

The workload is a large table reduced to a few thousand groups and joined with a tiny table, so each query should cost about one Parquet scan; extra scans, shuffles of full rows and spills come from how the query is written. We picked O1 and O4 because both compute statistics per bucket and then go back to every trip (O1 with a window, O4 with a join), so both shuffle whole rows, which the mandatory operations hardly do.

2 PIPELINE

We wrote the pipeline in Kotlin with the DataFrame API (agreed with the instructor instead of PySpark), which lets an operation mix Catalyst [1] queries with plain JVM code where that is faster (the driver-side windows of M3 v3, the columnar reader of M1 v3). Every operation implements the same `Analysis` interface and has numbered versions chosen on the command line, so each change is measured on its own against the baseline. M1 returns the cleaned DataFrame lazily and the other operations read from it; we do not cache it, since 142 M rows do not fit in 2 GB executors and each operation reads only its own columns. Every operation writes its result as Parquet (M1 writes its per-year cleaning statistics as CSV).

Figure 1 shows the physical plan of O1 (from `explain(mode="extended")`) before and after its key change. In v0 every trip passes through `Exchange hashpartitioning(PULocationID, hour)` and a `Sort` before the `Window`; in v3 the trips are only read and probed against a broadcast table built by a partial `HashAggregate`.

```
HashAggregate (+ Exchange,
   |   HashAggregate, write)
   +- BroadcastHashJoin
      :- Project
      :  +- Window
      :     +- Sort
      :        +- Exchange
      :           +- Filter
      :              +- Scan parquet
      +- BroadcastExchange
         +- Scan csv
```

```
HashAggregate (+ Exchange,
   |   HashAggregate, write)
   +- BroadcastHashJoin
      :- BroadcastHashJoin
      :  :- Filter
      :  :  +- Scan parquet
      :  +- BroadcastExchange
      :     +- HashAggregate
      :        +- Exchange
      :           +- HashAggregate
      :              +- Scan parquet
      +- BroadcastExchange
         +- Scan csv
```

Figure 1: Physical plan of O1 below the final aggregation and write, which are the same in both (Project nodes omitted): v0 with a Window over shuffled trips (left), v3 with a broadcast of the per-bucket statistics (right).

3 PROFILING AND BASELINE

Setup. We measured on the course's Spark standalone cluster (`docker/docker-compose.yml`, image `spark:python3`: Spark 4.2.0, Scala 2.13, OpenJDK 21) in Docker Desktop on a laptop with an Intel Core Ultra 7 256V (4 performance + 4 efficiency cores), 16 GB RAM, NVMe SSD and Windows 11. One master and two workers with 2 cores and 2 GB each; the driver runs in the master container (client mode) and the jar, data and output are mounted from Windows. Spark settings are the defaults (AQE on).

Methodology. `scripts/benchmark.sh` compiles the jar once and runs every version as a separate `spark-submit`, so no cache or JIT state carries over. Analytical operations run after M1 v3 and we count only their own jobs, with 20 s pauses between operations. Each version ran once, so when two versions have the same plan and shuffle we attribute a time difference to the run, not to the code. Our profiler reads the Spark REST API [3] (stages, tasks, CPU, GC, shuffle, spill, peak execution memory) into JSON, and `notebooks/benchmarks_report.ipynb` turns it into the tables and charts below. Every version produces the same output as the baseline unless stated.

What we found (Table 1):

- *Repeated scans.* M1 decodes every row twice (the per-year statistics and the second `count()`), and M3 three times, because `show()`, the count of flagged rows and the write are separate actions and nothing is cached. M3 is the slowest operation although its result has about 3,000 rows.
- *Shuffling whole rows.* The window of O1 shuffles every trip; with 2 GB executors the largest buckets spill 1.3 GB to disk and the

Table 1: Baseline (every operation at v0) on the cluster. "Heaviest stage" is the task time of the most expensive stage.

Op	wall (s)	stages	tasks	shuffle (MB)	spill (MB)	heaviest stage (task time)
M1	35.1	15	135	0	0	scan, 60 s (2 such)
M2	25.4	11	71	1.4	0	scan, 78 s
M3	97.7	44	254	2.5	0	scan, 129 s (3 such)
O1	80.5	16	195	1,515	1,319	window, 180 s
O4	68.0	18	247	1,799	0	shuffle write, 138 s
Total	**306.7**					

window stage is the most expensive of the baseline. O4 shuffles 1.8 GB to join the bucket classes back to the trips.

- *Many small stages.* M3 works on a few thousand rows after the aggregation, but each window adds an Exchange and a Sort (44 stages).
- *Little to gain.* M2 shuffles 1.4 MB: its plan was already close to one scan.
- *No skew.* Each scan task reads one file and the shuffle keys have thousands of values. The largest slowest/median task ratio is 2.9×, in the spilling window stage of O1 (1.3× once the window is gone), far from the 10× of real skew.

4 OPTIMISATION STUDY

Each subsection follows one operation version by version. Figure 2 shows how time and memory change between versions and Table 2 summarises all of them.

4.1 M1: one scan and a direct columnar reader

M1 v1: one scan. (a) The baseline scans the data several times and casts every column of every row. **(b)** All counters can come from one pass, and most columns already have the right type. **(c)** One aggregation computes the total, the five criteria and the valid rows; the schema is checked once and only wrong types are cast. **(d)** 1.83× faster, with a third of the jobs and half the CPU time.

M1 v2: bitmask per row. (a) Seven conditional sums each evaluate their own condition on every row. **(b)** One integer per row with a bit per criterion should be cheaper. **(c)** v2 counts per (year, bitmask), a few dozen groups, and the driver derives the counters. **(d)** 10 % less CPU, same time.

M1 v3: direct columnar reader. (a) Spark still turns every column batch into rows before counting. **(b)** Reading the column vectors ourselves should leave only the cost of decoding Parquet. **(c)** Each task reads the batches of one file, builds the bitmasks in plain loops and counts in an array; the driver adds the 12 arrays. **(d)** 2 stages, 13 tasks, no shuffle and less CPU, but the same time as v2, although locally it was the fastest version by far. About 40 % of the task time is spent waiting for the files, which are read through the mount from Windows, so the reading speed sets the time, not the CPU.

4.2 M2: little room above the scan

M2 was already bound by its scan; the best version is 1.24× faster than v0.

M2 v1: projection and broadcast. (a) The baseline reads every column and joins the zone table. **(b)** Selecting five columns and forcing a broadcast should read less. **(c)** v1 does both. **(d)** Same plan, stages and shuffle as v0: Catalyst was already pruning and broadcasting the 12 KB table. The run was slower only because one task took twice as long as any task of v0, which is noise of a single run.

M2 v2: no global sort. (a) The `orderBy` before the partitioned write adds a range-partitioning shuffle, but **(b)** `partitionBy(borough)` only needs rows grouped by borough. **(c)** v2 uses `repartition(borough)`. **(d)** Two stages fewer, half the shuffle, time close to v0.

M2 v3: aggregate before the join. (a) The zone join is applied to every trip. **(b)** Aggregating first by (zone, day, hour) shrinks its input to a few thousand rows. **(c)** v3 aggregates per zone, then joins and aggregates per borough. **(d)** A third of the peak memory and a fifth less time than v2.

M2 v4: no join. (a) v3 still has a join and a second aggregation. **(b)** Zone→borough is a 265-entry map, not a join. **(c)** v4 builds the map on the driver, packs (borough, day, hour) into 11 bits and aggregates once. **(d)** The fastest version; peak memory 1.4 GB→32 MB.

M2 v5–v7: less work per row. (a) v4 evaluates a `CASE WHEN` chain and `unix_timestamp` on every row. **(b)** Less work per row and fewer stages should help. **(c)** v5 uses an array lookup and divides once per group, v6 writes with `coalesce(1)`, v7 reads the CSV on the driver. **(d)** Each is a little slower than the one before: the removed stages were cheap, `coalesce(1)` puts the last step in one task and v7 uses more CPU. Removing stages only pays off when scheduling them is the cost.

4.3 M3: one scan, integer keys, windows on the driver

M3 v1: cache the result. (a) `show()`, the count of flagged rows and the write each read and aggregate the trips again. **(b)** If the small final table is cached, the later actions cost almost nothing. **(c)** v1 caches the result (about 3,000 rows) and broadcasts the zone table. **(d)** One heavy scan instead of three: 1.43× faster, under half the CPU. The cache keeps 200 shuffle partitions, so tasks go from 254 to about 1,000 and the reported peak memory to 6.4 GB for tiny data.

M3 v2: integer keys. (a) `date_format` builds a string for each of 146 M rows and the grouping key has strings. **(b)** Grouping by integers and formatting after the aggregation moves that work to 3,000 rows. **(c)** v2 groups by (`PULocationID`, year · 100 + month). **(d)** Another 1.27×, and CPU time halves again.

M3 v3: windows on the driver. (a) Each window still needs its own Exchange and Sort for 3,000 rows. **(b)** A loop on the driver should be cheaper than scheduling dozens of stages. **(c)** v3 collects the aggregated rows and computes the rank, the 3-month rolling share and the deviation flag (Welford [4]) in Kotlin. **(d)** 48→8 stages, about 1,000→38 tasks and 5.1× faster than v0, the largest gain of the project. We also found that the original flag could never fire (with the current month inside the 3-row frame, $|x - \bar{x}|/s \leq 1.155 < 1.5$), so all versions now use the three *previous* months.

4.4 O1: from a window to a broadcast

O1 v1 and v2: projection and broadcast. (a) The baseline carries all columns through several projections and joins the zone table.

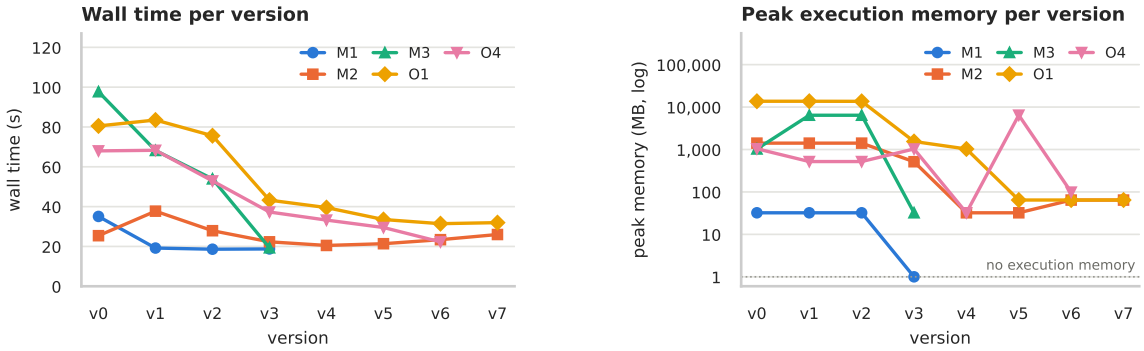


Figure 2: Wall time (left) and peak execution memory (right, log scale) of every version, on the cluster with the 2015 data. Memory is the sum of the task peaks per stage, as Spark reports it; M1 v3 bypasses Spark's operators and reports none.

Table 2: Summary of all optimisations on the cluster (2015 data, one run each; memory is the peak execution memory). The best versions (M1 v2, M2 v4, M3 v3, O1 v6, O4 v6) take 112.1 s together instead of 306.7 s for v0 (2.74×).

Op	ver	wall (s)	vs v0	mem (MB)
M1	v0	35.1	1.00	32
	v1	19.2	1.83	32
	v2	18.6	1.89	32
	v3	18.8	1.87	0
M2	v0	25.4	1.00	1,412
	v1	37.7	0.67	1,412
	v2	27.9	0.91	1,412
	v3	22.3	1.14	515
	v4	20.5	1.24	32
	v5	21.4	1.19	32
	v6	23.4	1.09	64
	v7	26.0	0.98	64

Op	ver	wall (s)	vs v0	mem (MB)
M3	v0	97.7	1.00	1,028
	v1	68.2	1.43	6,412
	v2	53.9	1.81	6,412
	v3	19.3	5.06	32
O1	v0	80.5	1.00	13,703
	v1	83.5	0.96	13,703
	v2	75.6	1.06	13,639
	v3	43.2	1.86	1,544
	v4	39.5	2.04	1,029
	v5	33.5	2.40	64
	v6	31.4	2.56	64
	v7	32.0	2.52	64

Op	ver	wall (s)	vs v0	mem (MB)
O4	v0	68.0	1.00	1,040
	v1	68.3	1.00	520
	v2	52.9	1.29	520
	v3	37.3	1.82	1,029
	v4	33.2	2.05	32
	v5	29.5	2.31	6,450
	v6	22.3	3.05	99
All	best	112.1	2.74	

(b) One projection and a forced broadcast should move less data. **(c)** v1 uses one `select`, v2 adds `broadcast(zones)`. **(d)** Same plan, shuffle and spill: Catalyst was already doing both, and the times are within single-run noise.

O1 v3: aggregation instead of a window. **(a)** A window cannot combine on the map side, so every trip is shuffled and large buckets spill. **(b)** μ and σ per bucket are an aggregation with at most 263×24 groups. **(c)** v3 uses `groupBy(zone, hour)`, which aggregates before the shuffle, and broadcasts the 6,312 thresholds back to the trips. **(d)** Shuffle 1.5 GB→3 MB, no spill, nine times less peak memory and 1.86× faster. The price, a second scan, costs far less than shuffling and spilling.

O1 v4 and v5: integer keys and an array instead of the join. **(a)** The broadcast join hashes and probes every trip, and the aggregation carries strings. **(b)** Integer keys and an array lookup should be cheaper. **(c)** v4 packs a 14-bit bucket key and an 8-bit rate key (borough resolved on the driver); v5 puts the thresholds in a 67 KB array read with `element_at`, so no join is left. **(d)** 10–15 % per step; after v5 peak memory is 64 MB.

O1 v6 and v7: less arithmetic per trip. **(a)** v5 updates a floating-point average per trip and converts every fare to per-km in both scans. **(b)** The rate can come from two integer counts, and $x > \mu+3\sigma$ is unchanged by scaling everything by 1/1.609. **(c)** v6 uses integer counters; v7 compares in fare per mile. **(d)** Both within noise of v5: O1 now costs about its two scans, 2.56× faster than v0.

4.5 O4: removing the shuffle, then the second scan

O4 v1: one projection. **(a)** Both branches share a wide DataFrame built with chained `withColumn`. **(b)** One `select` of seven columns should make the shuffled rows smaller. **(c)** v1 projects them and derives hour and month in one step. **(d)** 10 % less shuffle and half the peak memory, same time: the 1.6 GB shuffle of every trip is still there.

O4 v2: broadcast the bucket classes. **(a)** The join of trips and classified buckets is a sort-merge join that shuffles all 142 M trips. **(b)** The planner cannot know the aggregation has at most 6,312 rows, but we do. **(c)** v2 wraps the classes in `broadcast()`. **(d)** Shuffle 1.6 GB→1.7 MB and a quarter less time.

O4 v3: integer keys. **(a)** The join and aggregations use a two-column key and strings (`date_format` for every trip). **(b)** Integers move the string work to the 48 output rows. **(c)** v3 packs (zone, hour) into 14 bits and the zone type into 2. **(d)** 30 % less CPU, 1.42× over v2.

O4 v4: an array instead of the join. **(a)** The broadcast join still hashes every trip and three floating-point averages are updated per

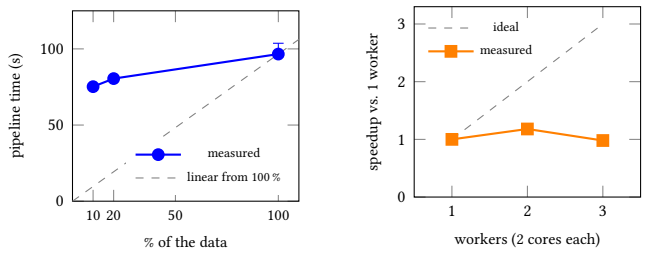


Figure 3: Optimised pipeline on the cluster, 2015 data (median of 3, bars show the range). Left: time vs. fraction of the data (2 workers). Right: speedup with 1, 2 and 3 workers on the full data.

row. **(b)** An array indexed by the key and integer counters should be cheaper. **(c)** v4 uses integer sums and an 8 KB array of zone types read with `element_at`. **(d)** 10 % faster; peak memory 1 GB→32 MB, as no hash table is built.

O4 v5: one scan. **(a)** Every version so far reads the data twice: to classify the buckets and to compute the monthly statistics. **(b)** Trips with the same (month, zone, hour) share a class, and counts, sums and sums of squares add up, so mean and standard deviation can come from about 38,000 partial groups. **(c)** v5 aggregates the trips once into them, caches them and finishes with two small Spark jobs. **(d)** A quarter less CPU and 11 % less time, but 22 stages, 1,500 tasks and 6.4 GB of reported peak memory, as in M3 v1.

O4 v6: finish on the driver. **(a)** v5 spends stages and tasks to reduce 38,000 rows. **(b)** As in M3 v3, plain code on the driver should be cheaper. **(c)** v6 packs (month, zone, hour) into one integer, aggregates the trips once, collects the 38,000 rows (2 MB) and finishes in Kotlin. **(d)** 6 stages, 38 tasks, 99 MB and 3.05× faster than v0; results match v0 up to the last digits of the standard deviation.

4.6 Summary

Table 2 lists every version: the best ones take 112.1 s instead of 306.7 s (2.74×). The largest gains come from removing work: extra scans, shuffles of whole rows and stages for a few thousand rows (M1 v1, M3 v3, O1 v3, O4 v2, O4 v6). Memory falls when a join or hash table goes away (M2 v4, O1 v5, O4 v4), while caching a small result in 200 partitions shows up as a memory peak even when it saves time (M3 v1, O4 v5). What Catalyst already did on its own (projections and broadcasts of the zone table) gave nothing.

5 ANALYSIS

We ran the optimised pipeline (the best version of each operation) with `scripts/cluster_bench/scaling_2015.sh`, which calls the same `benchmark.sh` three times per condition; we report the median and give the range in Figure 3.

Dataset scale. The time grows much less than the data. With 10 % of the trips the pipeline takes 75.2 s, 78 % of the 96.6 s of the full data, and with 20 % it takes 83 %: ten times less data saves only a fifth of the time. A straight line through the three points gives about 73 s that do not depend on the fraction and 24 s that do. Two things make the fixed part so large. Scheduling the jobs and stages, reading the zone table and the work on the driver cost the same for any input. And `--sample` draws the rows *after* reading, so every Parquet file is still opened, read through the mount and decoded; only the work after the scan shrinks. That is why O4, which still aggregates every trip, drops by 31 %, while M1, which is almost only the read, drops by 18 %.

Cluster size. With 2 workers the pipeline is 1.18× faster than with one (efficiency 59 %), and with 3 it is no faster than with one (0.98×, efficiency 33 %), with the widest spread between runs. The deviation from the ideal line has three causes. The fixed cost on the driver and in scheduling does not shrink with more executors. The scans have one task per file (13 tasks), so with 4 or 6 cores the last wave is almost empty. Most important, all workers are containers on the same laptop and share its 8 cores (half of them efficiency cores), its memory bandwidth and the mount through which every file is read. The executor CPU time for the same work grows from 157 to 224 and 309 s with 1, 2 and 3 workers: the extra executors mostly compete for the same hardware. On real machines, each with its own cores and disk, we would expect the speedup to follow the scans much more closely.

6 DISCUSSION

What is left. Each operation now costs roughly the scans it needs: one pass for M1, M2, M3 and O4, two for O1. On the cluster the next limit is reading the files through the mount. Every operation still reads the same files again; materialising the cleaned data once (a cache with enough memory, or a narrower Parquet file) would fix this.

Limitations. Each version of Section 4 ran once, so a slow task can move a result (M2 v1); we only draw conclusions when the plan metrics change as well. The Docker cluster shares the laptop's CPUs, so it shows the overhead of a cluster but not what real machines would give. Peak memory is the sum of task peaks per stage, too high for stages with many tasks (M3 v1, O4 v5), and on the cluster Spark does not report Parquet input bytes, so we reason about scans from the plans. **With more time** we would repeat every version three times, run on real machines (the DI cluster) and measure the cost of materialising the output of M1.

ACKNOWLEDGMENTS

We used two AI tools during the project, Claude Code (Anthropic) and Antigravity (Google), mostly to help with research, with the code and with writing this report. We ran and checked all the measurements ourselves.

INDIVIDUAL CONTRIBUTIONS

Member	Work	%
Diogo Oliveira	M1, M3, benchmark, report	[TODO: %]
André Nunes	M2, O1, O4, profiler, Docker benchmark	[TODO: %]

REFERENCES

[1] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1383–1394.

[2] NYC Taxi & Limousine Commission. 2024. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page. Accessed November 2026.

[3] The Apache Software Foundation. 2024. Apache Spark 3.5.3 Documentation: SQL Performance Tuning and Monitoring and Instrumentation. https://spark.apache.org/docs/3.5.3/.

[4] B. P. Welford. 1962. Note on a Method for Calculating Corrected Sums of Squares and Products. *Technometrics* 4, 3 (1962), 419–420.